

Tipi di Dati

Tipi di Dati

I tipi di dati fondamentali del C++: int, float, double, char e bool.

Perché esistono i tipi di dati?

Immagina di avere dei cassetti in un armadio. Un cassetto è per i calzini, uno per le magliette, uno per i pantaloni. Non metteresti dei pantaloni nel cassetto dei calzini.

In C++ funziona allo stesso modo: ogni variabile ha un **tipo** che stabilisce che genere di valore può contenere. A differenza di Python, non puoi mettere qualsiasi cosa in qualsiasi variabile: devi dichiarare il tipo in anticipo.

Questo rende il codice più sicuro e veloce.

Numeri interi: `int`

`int` (abbreviazione di *integer*, intero) contiene numeri senza virgola, sia positivi che negativi:

```
int eta = 16;  
int temperatura = -5;  
int anno = 2024;  
int zero = 0;
```

Un `int` occupa 4 byte di memoria e può contenere valori da circa -2 miliardi a +2 miliardi. Per la maggior parte dei programmi, è più che sufficiente.

Se hai bisogno di numeri molto grandi o molto piccoli, esistono varianti:

```
short int piccolo = 32000; // massimo circa 32.000  
long long enorme = 9000000000000LL; // fino a 9 miliardi di miliardi
```

Numeri decimali: `float` e `double`

Per numeri con la virgola (come 3.14 o 1.75) hai due opzioni.

`float`

Occupa 4 byte. Preciso fino a circa 7 cifre. Metti la `f` alla fine del numero per indicare che è un float:

```
float temperatura = 36.6f;  
float pi = 3.14159f;
```

`double`

Occupa 8 byte. Preciso fino a circa 15-16 cifre. È quello più usato:

```
double altezza = 1.75;  
double pi = 3.14159265358979;
```

Quale usare?

Usa sempre `double`, a meno che tu non abbia un motivo specifico per usare `float`. Il `double` è più preciso e il tipo preferito in C++ per i numeri decimali:

```
// Possibile problema con float  
float a = 0.1f + 0.2f;  
cout << a; // potrebbe stampare 0.3000001 invece di 0.3  
  
// Meglio con double  
double b = 0.1 + 0.2;  
cout << b; // molto più preciso
```

Singolo carattere: **char**

char (abbreviazione di *character*, carattere) contiene un singolo carattere. I caratteri si scrivono tra **apici singoli** `'...'`:

```
char lettera = 'A';  
char cifra = '7';  
char spazio = ' ';
```

Internamente, ogni carattere è memorizzato come un numero (il suo codice ASCII). Per esempio, `'A'` corrisponde al numero 65:

```
char c = 'A';  
cout << c;           // stampa: A  
cout << (int)c;       // stampa: 65 (il codice numerico di 'A')
```

Vero o falso: **bool**

bool (abbreviazione di *boolean*, booleano) può contenere solo due valori: **true** (vero) o **false** (falso):

```
bool maggiorenne = true;  
bool haPatente = false;  
bool risultato = (5 > 3); // true, perché 5 è maggiore di 3
```

Quando stampi un **bool**, di default viene mostrato come **1** (true) o **0** (false). Per stampare le parole **true** e **false**:

```
bool x = true;  
cout << x;           // stampa: 1  
cout << boolalpha << x; // stampa: true
```

Quanto spazio occupano?

Puoi controllare quanti byte occupa un tipo con `sizeof` :

```
cout << sizeof(int) << endl;    // 4
cout << sizeof(double) << endl; // 8
cout << sizeof(char) << endl;   // 1
cout << sizeof(bool) << endl;   // 1
```

Riepilogo

Tipo	Spazio	Valori
<code>int</code>	4 byte	numeri interi, da -2 miliardi a +2 miliardi
<code>float</code>	4 byte	decimali, ~7 cifre significative
<code>double</code>	8 byte	decimali, ~15 cifre significative
<code>char</code>	1 byte	un singolo carattere
<code>bool</code>	1 byte	solo <code>true</code> oppure <code>false</code>

Convertire tra tipi

A volte hai bisogno di convertire un valore da un tipo all'altro.

La **conversione implicita** avviene automaticamente quando è sicura:

```
int x = 5;
double y = x;    // int viene convertito in double automaticamente
cout << y;       // 5.0
```

La **conversione esplicita** (chiamata *casting*) si usa quando vuoi forzare una conversione:

```
double pi = 3.14159;
int parteIntera = (int)pi; // la parte decimale viene scartata
cout << parteIntera;      // 3 (non arrotondato, solo troncato)
```

Attenzione alla **divisione tra interi**: quando dividi due `int`, il risultato è sempre un intero (la parte decimale viene persa):

```
int x = 5, y = 2;
int risultato = x / y;      // risultato = 2, non 2.5!
double risultato2 = (double)x / y; // risultato2 = 2.5 (corretto)
```

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    // Una variabile per ogni tipo principale
    int eta = 16;
    double altezza = 1.75;
    char iniziale = 'A';
    bool maggiorenne = false;

    cout << "Età (int): " << eta << endl;
    cout << "Altezza (double): " << altezza << endl;
    cout << "Iniziale (char): " << iniziale << endl;
    cout << "Maggiorenne (bool): " << boolalpha << maggiorenne << endl;

    return 0;
}
```

Output:

```
Età (int): 16
Altezza (double): 1.75
Iniziale (char): A
Maggiorenne (bool): false
```